

Iteratori in C++

Iteratori in C++

Introduzione agli iteratori della STL in C++.

Cos'è un iteratore?

Gli algoritmi della STL (come `sort`, `find`, `count`) non lavorano direttamente con i contenitori. Lavorano con degli oggetti speciali chiamati **iteratori** che indicano "da dove a dove" operare.

Capire gli iteratori ti permette di usare tutti gli algoritmi della STL e di scorrere i contenitori in modi più sofisticati.

Cosa sono gli iteratori

Un iteratore è come un **segnalibro**: punta a un elemento del contenitore e può spostarsi all'elemento successivo (o precedente).

Pensa a un iteratore come a un dito che scorre su una lista: parte dall'inizio, si sposta verso destra elemento per elemento, e si ferma alla fine.

I metodi `begin()` e `end()`

Ogni contenitore STL ha due metodi fondamentali:

- `begin()` — restituisce un iteratore che punta al **primo** elemento
- `end()` — restituisce un iteratore che punta **dopo** l'ultimo elemento

```
vector<int> v = {10, 20, 30, 40, 50};

// begin() punta a 10
// end() punta a una posizione immaginaria dopo il 50
```

`end()` non punta a un elemento reale: è solo un segnaposto che indica "siamo arrivati alla fine".

Usare un iteratore

Dichiara un iteratore con `auto` (il compilatore capisce il tipo da solo):

```
vector<int> v = {10, 20, 30, 40, 50};

auto it = v.begin();           // it punta al primo elemento (10)
cout << *it << endl;          // 10 - il * "legge" il valore puntato

++it;                          // sposta l'iteratore al prossimo elemento
cout << *it << endl;          // 20

++it;
cout << *it << endl;          // 30
```

L'operatore `*` davanti all'iteratore "dereferenzia": legge il valore dell'elemento puntato.

Scorrere con un ciclo

```
vector<int> v = {10, 20, 30, 40, 50};

// Ciclo while con iteratore
auto it = v.begin();
while (it != v.end()) {
    cout << *it << " ";
    ++it; // sposta al prossimo
}
// Output: 10 20 30 40 50
```

```
// Ciclo for con iteratore - forma compatta
for (auto it = v.begin(); it != v.end(); ++it) {
    cout << *it << " ";
}
```

:::note Il range-based for (`for (int n : v)`) usa gli iteratori internamente. Quindi di solito non devi scrivere esplicitamente gli iteratori, ma capire come funzionano ti aiuta a usare la STL. :::

Iteratori che non modificano: `cbegin()` e `cend()`

Se vuoi scorrere gli elementi senza modificarli, usa `cbegin()` e `cend()` (la `c` sta per "constant"):

```
vector<int> v = {1, 2, 3};

for (auto it = v.cbegin(); it != v.cend(); ++it) {
    cout << *it << " ";
    // *it = 99; // ERRORE - con l'iteratore costante non puoi modificare
}
```

Scorrere al contrario: `rbegin()` e `rend()`

```
vector<int> v = {1, 2, 3, 4, 5};

for (auto it = v.rbegin(); it != v.rend(); ++it) {
    cout << *it << " ";
}
// Output: 5 4 3 2 1
```

`rbegin()` punta all'ultimo elemento, `rend()` punta "prima" del primo.

Iteratori e algoritmi STL

Gli iteratori sono il modo in cui colleghi i tuoi dati agli algoritmi:

```

#include <algorithm>
#include <vector>
using namespace std;

vector<int> v = {5, 2, 8, 1, 9, 3};

// Ordina tutti gli elementi da begin a end
sort(v.begin(), v.end());

// Cerca il valore 5 e restituisce un iteratore all'elemento trovato
auto it = find(v.begin(), v.end(), 5);
if (it != v.end()) {
    // it - v.begin() calcola la posizione (distanza dall'inizio)
    cout << "Trovato 5 alla posizione " << (it - v.begin()) << endl;
}

// Rimuove l'elemento alla posizione 2
v.erase(v.begin() + 2);

```

Se `find` non trova il valore, restituisce `v.end()`. Ecco perché il controllo `if (it != v.end())` è importante.

Esempio pratico

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> numeri = {3, 1, 4, 1, 5, 9, 2, 6, 5, 3};

    // Stampa la lista originale usando un iteratore costante
    cout << "Originale: ";
    for (auto it = numeri.cbegin(); it != numeri.cend(); ++it) {
        cout << *it << " ";
    }
    cout << endl;

    // Ordiniamo il vector
    sort(numeri.begin(), numeri.end());
    cout << "Ordinato: ";
    for (int n : numeri) cout << n << " ";
    cout << endl;

    // Stampiamo al contrario con l'iteratore inverso
    cout << "Inverso: ";
    for (auto it = numeri.rbegin(); it != numeri.rend(); ++it) {
        cout << *it << " ";
    }
    cout << endl;

    // unique rimuove i duplicati consecutivi – richiede il vector ordinato
    auto fine = unique(numeri.begin(), numeri.end());
    numeri.erase(fine, numeri.end()); // rimuoviamo i "resti" lasciati da
unique
    cout << "Senza duplicati: ";
    for (int n : numeri) cout << n << " ";
    cout << endl;

    return 0;
}
```

Output:

```
Originale: 3 1 4 1 5 9 2 6 5 3  
Ordinato: 1 1 2 3 3 4 5 5 6 9  
Inverso: 9 6 5 5 4 3 3 2 1 1  
Senza duplicati: 1 2 3 4 5 6 9
```